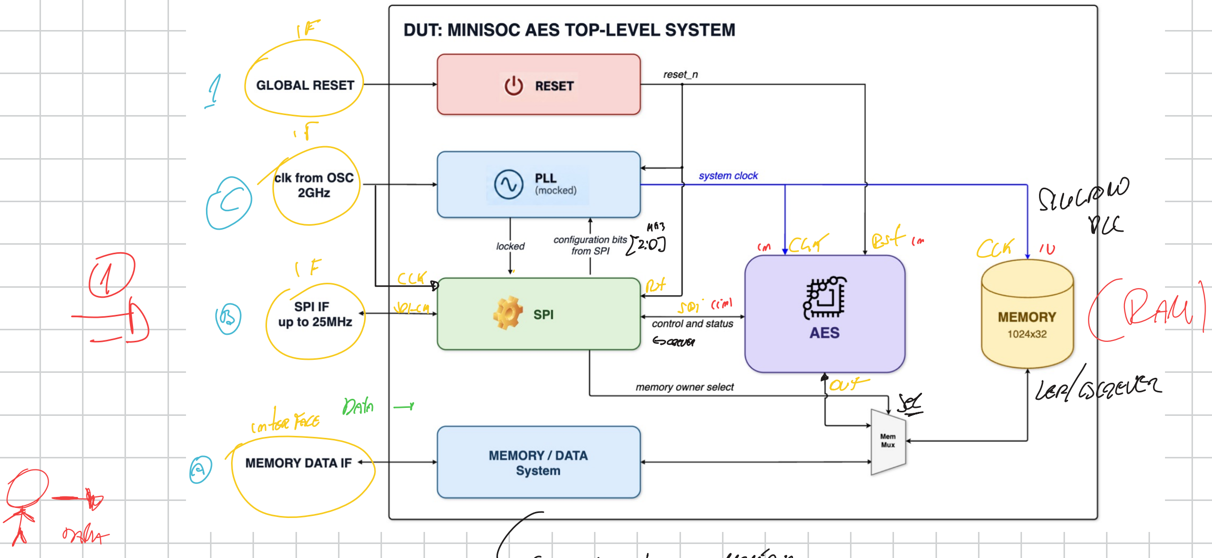
DitL

(Day in the Life)


DUT: MINISOC AES TOP-LEVEL SYSTEM
IF
1
GLOBAL RESET
RESET
reset_n
IF
C
clk from OSC 2GHz
PLL (mocked)
system clock
Sincroniza PLL
locked
configuration bits from SPI
MR3 [2:0]
cm
CLK
Rst
cm
CLK
IN
IF
①
→
B
SPI IF up to 25MHz
CLK
spi_clk
SPI
Rst
SPI (cim)
control and status
escrever
AES
MEMORY 1024x32
(RAM)
ler/escrever
memory owner select
out
Sel
Mem Mux
interface
Data →
@
MEMORY DATA IF
MEMORY / DATA System
data

Locken e Reset

Caminho externo memória

SPI

① O SPI é sincronizado pelo clk (2GHz - always-on (independente-OSC))
ou seja, ele está sempre operando mesmo que ele não esteja locked.

• "Ele realiza o controle do AES e memória" + definindo os parâmetros de operação
↳ controla a seleção do mux (memory e AES)

O SPI configura o PLL onde em locked (0) - sys_clk = stop
o AES não acessa a mem

locked (1) - sys_clk - liberado
AES pode começar. 0

② Memória

O SPI realiza o controle da seleção de memória entre a memória externa (memory/data) e AES, como também em qual clock ele vai atuar.

Onde:

- Mem_sel_AES = 0 ; memory/data controla a porta da memória (mem clk) cram
- Mem_sel_AES = 1 ; AES controla a porta da memória (sys clk)

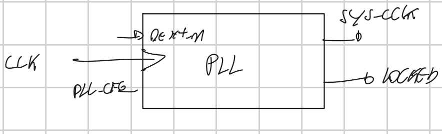
apenas um acessa por vez

↳ Obs: a seleção de Mem_sel_AES se mantém até o AES estiver em idle - se a interface externa não estiver realizando uma transação.
↳ AES_busy = 1

③ 'PLL (mock)' (clk, en, selector[1:0], locked, en)

↳ basicamente recebe o clk, porém ele divide conforme a seleção.
↳ o PLL libera o locked (simula) conforme o comando do SPI
↳ liberando o sys_clock para RAM e o AES.
↳ o locked se mantém ativo,
↳ só pode mudar quando o AES terminar operação.

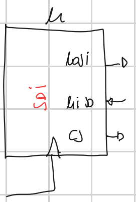
↳ o SPI pode alterar o divisor durante operação!


CLK
Reset_n
PLL
PLL_CFG
SYS_CLK
LOCKED

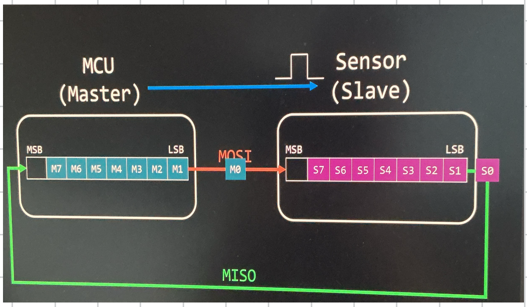
Convencionalmente não deve acontecer e se sim junto com que locked = 0.

Observações (SPI)

↳ O SPI é um protocolo de comunicação (M/S)
↳ pode enviar e receber ao mesmo tempo
↳ comunicação síncrona
↳ 4 fios (spi_clk, MISO, MOSI, CS)
↳ MOSI → Most out slaved = M → S (posedge)
↳ MISO → Most in slaved = S → M (negedge)


M
MOSI
MISO
CS
SPI
SPI_CLK

	(polaridade)	(fase)
Mode	CPOL	CPHA
0	0	0
1	0	1
2	1	0
3	1	1

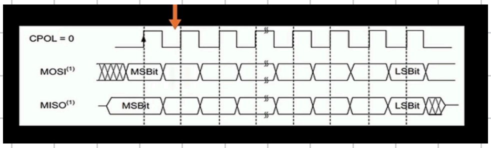

MCU (Master)
Sensor (Slave)
MSB
LSB
M7 M6 M5 M4 M3 M2 M1
MOSI
M0
MSB
LSB
S7 S6 S5 S4 S3 S2 S1
S0
MISO

MISO
(M) LSB → MSB (S)

MOSI
(S) LSB → MSB (M)

Mode 0

CPOL = 0 → clock low → comunicação inativa
clock high → " ativa
CPHA = 0 → lido na primeira borda (leem e escrevem)
2ª borda - negedge → deslocamento


CPOL = 0
MOSI(1)
MSBit
LSBit
MISO(1)
MSBit
LSBit

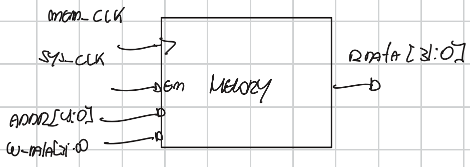
① RAM (MEMORY)

↳ A RAM comporta 256 blocos AES de 128 bits

↳ 1024 x 32 , 1024 posições de memória do tamanho de 32 bits = 1024 x 32 = 32 768 bits = 4KiB

↳ O acesso à memória deve acontecer uma vez por vez, a troca deve ocorrer quando o AES está ocioso e sem operação

mem_sel (AES) = 0
= 1


mem_CLK
sys_CLK
en
MEMORY
ADDR[9:0]
w_data[31:0]
rdata[31:0]

mem_i_wr_en = 1 ; write mem_i_data em mem_i_addr (mem_clk posedge)

mem_i_wr_en = 0 ; read mem_i_data em em mem_o_data (mem_clk

③ Reset (ativo em nível baixo)

↳ O reset_n é aplicado aos blocos de controle (voltando ao estado inicial)

↳ O reset se mantém por alguns ciclos

ε O reset interno do AES permanece ativo enquanto o PLL não está sincronizado e por três ciclos adicionais de sistema após o lock.

Depois disso AES é liberado para operar

Locked = 0 → reset

↳ A perda de locked reativa o reset?

↳ Interromper o AES

↳ Reinicia o AES

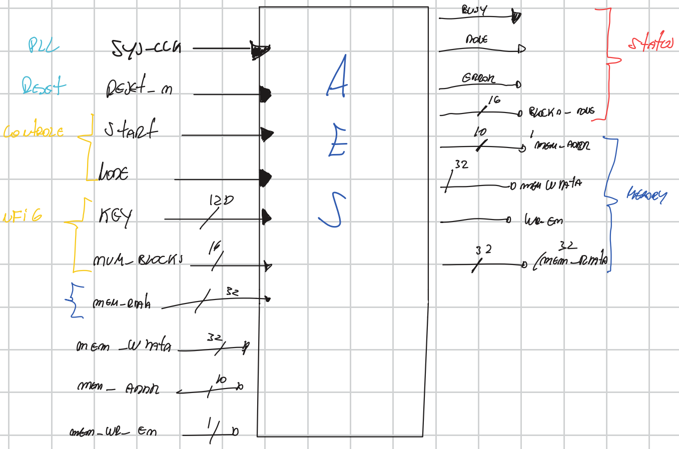
↳ Cancela uma operação AES?

↳ Como garantir que uma transmissão não corrompa

AES

↳ Bloco de 128 bits e key de 128 bits

↳ AES_BUSY = 1 ⇒ SPI


PLL
Reset
SPI
Controle
SPI
Config
sys_clk
reset_n
start
mode
KEY
128
num_blocks
16
mem_rdata
32
mem_wdata
32
mem_addr
10
mem_wr_en
1
A
E
S
BUSY
DONE
ERROR
16
BLOCKS_DONE
10
mem_ADDR
32
mem_wdata
wr_en
32
32
mem_rdata
status
Memory

↳ O bloco de 128 bits: quantidade de dados processados (16 bytes)

↳ Chave 128 bits: quantidade de informação secreta utilizada na criptografia.

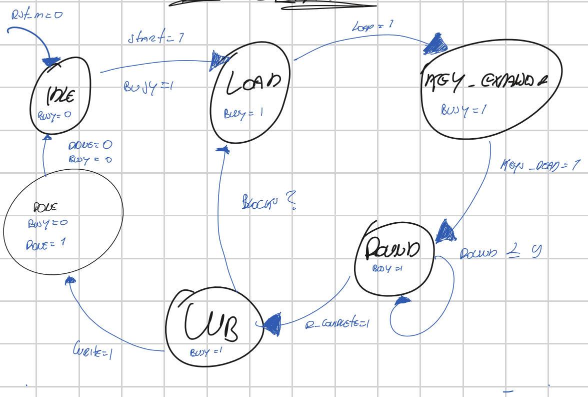
↳ O AES não opera em uma única transação (são executados 10 rounds)

↳ Ele pode ser expandido (key expansion)

1 chave inicial → 10 chaves

OBS: 10 rounds ≠ 10 clocks

FSM - Operation


rst_n=0
IDLE
BUSY=0
start=1
BUSY=1
LOAD
BUSY=1
LOAD=1
KEY_EXPAND
BUSY=1
KEYS_READY=1
ROUND
BUSY=1
ROUND ≤ 9
R_COMPLETE=1
WB
BUSY=1
BLOCKS ?
WRITE=1
DONE
BUSY=0
DONE=1
DONE=0
BUSY=0

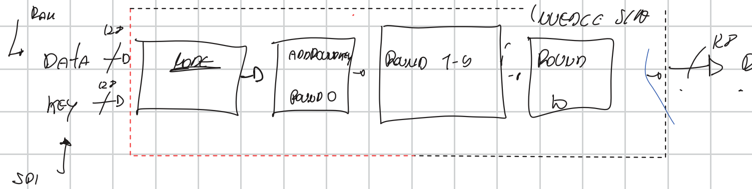
OBS: Cipher → criptografia plaintext → ciphertext

OBS: Inverse cipher → chave para text original

OBS: Como funciona de fato. (Não como algoritmo / hardware / SoC)

↳ Algoritmo → SubBytes(), ShiftRows(), MixColumns(), AddRoundKey/sci·, no ShiftRows().

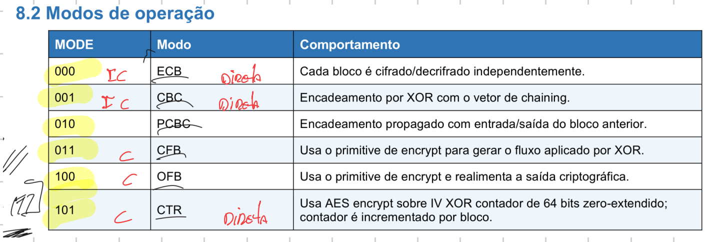
↳ Carregam o dado (128) e key (128) → Cypher


RAM
DATA
128
KEY
128
SPI
LOAD
ADDROUNDKEY
ROUND 0
ROUND 1-9
ROUND
10
128
Result

Operações: ECB – blocos independentes. Saída independentes

CBC – blocos encadeados (IV)

Modos de operação



8.2 Modos de operação

MODE	Modo	Comportamento
000 IC	ECB Direta	Cada bloco é cifrado/decifrado independentemente.
001 IC	CBC Direta	Encadeamento por XOR com o vetor de chaining.
010	PCBC	Encadeamento propagado com entrada/saída do bloco anterior.
011 C	CFB	Usa o primitive de encrypt para gerar o fluxo aplicado por XOR.
100 C	OFB	Usa o primitive de encrypt e realimenta a saída criptográfica.
101 C	CTR Direta	Usa AES encrypt sobre IV XOR contador de 64 bits zero-extendido; contador é incrementado por bloco.

ECB: Modo mais simples recebe key e blocos → resultado = P1 → AES → C1
P2 → AES → C2

CBC: Cada bloco depende do resultado criptografado anterior = P1 → XOR + IV → AES → C1
(procedural) P2 → XOR + C1 → AES → C2

P CBC: mesma coisa que CBC + utilizando o original = P1 → XOR + IV → AES → C1
P2 → XOR (P1 XOR C1) → AES → C2

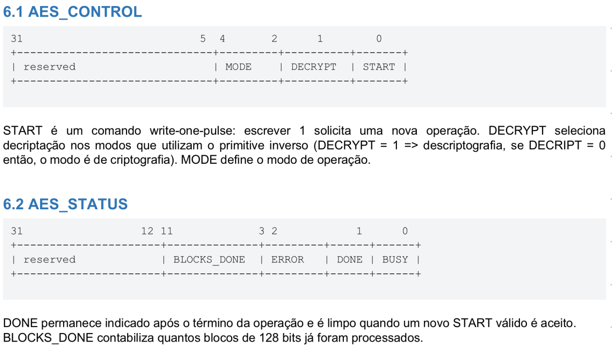
→ CFB: AES criptografa um valor de feedback, e o resultado é combinado com os dados usando XOR.
IV → AES → XOR + P1 → C1
C1 → AES → XOR + P2 → C2

→ OFB: AES gera uma sequência de bits independente do conteúdo mensagem.
→ IV → AES → S1 → XOR com Pi → Ci
AES → S2 → XOR

→ CTR: IV + Count → AES → XOR + Pi → Ci

SPI configura o PLL, sys_clk é liberado
O reset interno é liberado após 3 ciclos clock (sys)
SPI configura o AES enviando start, mode, key, num_blocks —
SPI libera a memória para o AES selecionando mem_sel, AES assume o controle da
SPI envia start
AES começa o processamento (busy = 1)
Lê 4 palavras de 32 bits na RAM para formar um bloco
Chave expande em 14 round keys
Executa 10 rounds do AES-128
Escreve resultado de 128 bits na memória em 4 de 32
repete o processo conforme número

AES finaliza → busy - OFF
done - ON
block done - 0



6.1 AES_CONTROL

```
31                           5  4        2        1        0
+----------------------------+---------+---------+-------+
| reserved                   | MODE    | DECRYPT | START |
+----------------------------+---------+---------+-------+
```

START é um comando write-one-pulse: escrever 1 solicita uma nova operação. DECRYPT seleciona decriptação nos modos que utilizam o primitive inverso (DECRYPT = 1 => descriptografia, se DECRIPT = 0 então, o modo é de criptografia). MODE define o modo de operação.

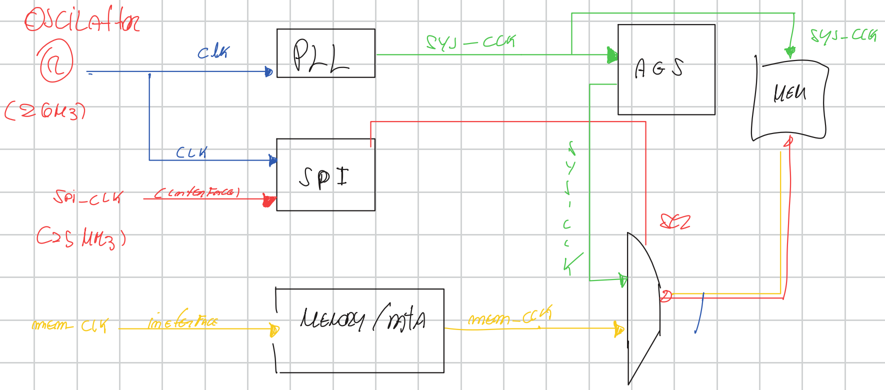
6.2 AES_STATUS

```
31                   12 11            3 2          1      0
+---------------------+--------------+---------+------+------+
| reserved            | BLOCKS_DONE  | ERROR   | DONE | BUSY |
+---------------------+--------------+---------+------+------+
```

DONE permanece indicado após o término da operação e é limpo quando um novo START válido é aceito. BLOCKS_DONE contabiliza quantos blocos de 128 bits já foram processados.

mem_sel_AES = 0, a interface memory lê o dado processado

Entendendo a árvore de clock



CLK (≥6MHz) - always_on
↳ funciona independente do PLL

SPI_CLK (≤25 MHz) → interface

Sys_clk (≥6MHz - 250 MHz) → interno (PLL)
↳ só é liberado quando locked
↳ controlado via SPI

Mem_clk (≤200MHz) →

Clock com ritmos e formas diferentes, um aviso (toggle) muito curto de fora "ato" "on"

Dit2

① SPI configurado AES e memory
② Memória disponível
③ PLL locked
④ AES operando (start)
⑤ Escrita na memória
⑥ Reset ?

① SPI configurado AES e memory
② PLL locked
③ Reset interno é liberado
④ Memória disponível, memória carregada → RAM (sel_aes=0)
○ SPI libera memória para o AES
○ SPI start
○ AES process
○ AES finished
○ SPI consulta status
○ Memória mostra o dado.

○ O que acontece memory_data escreve quatro palavras de 32 bits na RAM, o AES processa os bits e ao terminar, a interface então lê o dado?

Rotina padrão

① Inicialização
Reset global → SPI operacional
② Configuração do PLL
SPI configura locked = 1 → Reset interno liberado
③ Carregamento da RAM
mem_sel_aes=0 → memory data escreve na RAM
④ Configurar AES
SPI → {key, mode, num_blocks}
⑤ Processamento
mem_sel_aes = 1 → start (1) → busy = 1
⑥ Finalização
done = 1 → busy = 0
⑦ Leitura do resultado
mem_sel_aes = 0 → memory data (read)

Rotinas possíveis

	○ O que muda?	○ O que esperar
Criptografia simples	ECB mode	→ operação básica funcionando
Descriptografia	ECB	→ recupera corretamente?
Múltiplos blocos	num_blocks > 1	→ processamento sequencial e blocos novos?
Modos encadeados	CBC ou PCBC	→ uso de IV e bloco?
Modo CTR	Contador e cifra	→ geração de saída
Mudança de clock	Divisor de clk	→ operação com diferentes freq
Operações consecutivas	Start e done	→ quando o AES sem usar o reset
Configurações durante busy	SPI escreve AES	→ bloqueio das configurações protegido
Locked perdido	PLL perde sinal	→ ?
Reset durante operação	rst_n=0; busy=1	→ reinicia?

Operações

② Rotina contínua

① Criptografar
② Descriptografar
③ Alterar a chave
④ Executar operação (sem reiniciar)

③ Operação com restrições

① Iniciar uma criptografia com múltiplos
② Alterar CFG durante o busy on
③ SPI bloqueia, ou então o que

Testes / operações para observação

Ⓐ Operação normal → resultados (verificar todas as operações)
Ⓑ Operações com múltiplos blocos
Ⓒ Fazer todas as operações utilizando outras freq do PLL
Ⓓ O SPI bloqueia durante busy = 1 ?
Ⓔ Baixar o clock durante operação

A nível de SoC

Ⓒ Carregar toda a memória antes de habilitar o PLL
Ⓓ Habilitar o PLL sem carregar nada na memória?
Ⓔ Operar várias vezes (done=1) sem aplicar o reset.
○ Aplicar o reset aleatório? (existe algum padrão de reset)?

Pode variar a frequência durante operação? (durante locked)
○ O que acontece

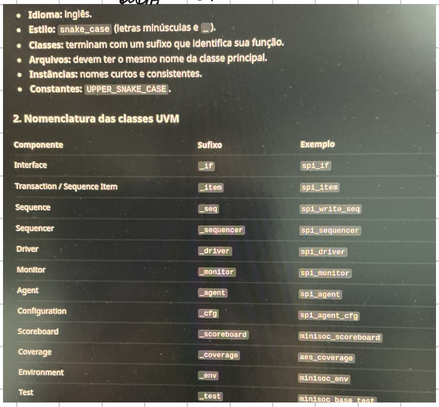
Cover

rst_n = [0, 1] → pode variar o reset
spi_cs_n = [0, 1]
mem_i_data = [0 – 32'h FFFFFFFF]   (7 6 5 4 3 2 1 0)
mem_i_addr = [0 – [0 – 1023]
mem_o_data = [0 – 32'hFFFFFFFF]

Day 2

Arquitetura.

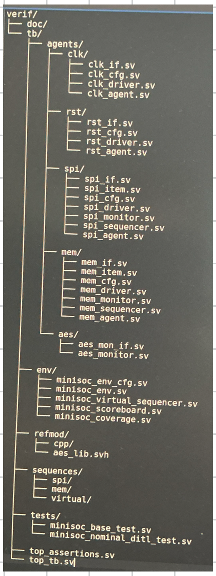
Regra nomenclatura



- **Idioma:** inglês.
- **Estilo:** `snake_case` (letras minúsculas e `_`).
- **Classes:** terminam com um sufixo que identifica sua função.
- **Arquivos:** devem ter o mesmo nome da classe principal.
- **Instâncias:** nomes curtos e consistentes.
- **Constantes:** `UPPER_SNAKE_CASE`.

2. Nomenclatura das classes UVM

Componente	Sufixo	Exemplo
Interface	`_if`	`spi_if`
Transaction / Sequence Item	`_item`	`spi_item`
Sequence	`_seq`	`spi_write_seq`
Sequencer	`_sequencer`	`spi_sequencer`
Driver	`_driver`	`spi_driver`
Monitor	`_monitor`	`spi_monitor`
Agent	`_agent`	`spi_agent`
Configuration	`_cfg`	`spi_agent_cfg`
Scoreboard	`_scoreboard`	`minisoc_scoreboard`
Coverage	`_coverage`	`aes_coverage`
Environment	`_env`	`minisoc_env`
Test	`_test`	`minisoc_base_test`



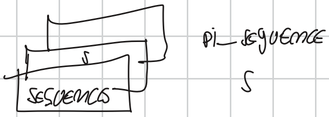
```
verif/
├── doc/
└── tb/
    ├── agents/
    │   ├── clk/
    │   │   ├── clk_if.sv
    │   │   ├── clk_cfg.sv
    │   │   ├── clk_driver.sv
    │   │   └── clk_agent.sv
    │   │
    │   ├── rst/
    │   │   ├── rst_if.sv
    │   │   ├── rst_cfg.sv
    │   │   ├── rst_driver.sv
    │   │   └── rst_agent.sv
    │   │
    │   ├── spi/
    │   │   ├── spi_if.sv
    │   │   ├── spi_item.sv
    │   │   ├── spi_cfg.sv
    │   │   ├── spi_driver.sv
    │   │   ├── spi_monitor.sv
    │   │   ├── spi_sequencer.sv
    │   │   └── spi_agent.sv
    │   │
    │   ├── mem/
    │   │   ├── mem_if.sv
    │   │   ├── mem_item.sv
    │   │   ├── mem_cfg.sv
    │   │   ├── mem_driver.sv
    │   │   ├── mem_monitor.sv
    │   │   ├── mem_sequencer.sv
    │   │   └── mem_agent.sv
    │   │
    │   └── aes/
    │       ├── aes_mon_if.sv
    │       └── aes_monitor.sv
    │
    ├── env/
    │   ├── minisoc_env_cfg.sv
    │   ├── minisoc_env.sv
    │   ├── minisoc_virtual_sequencer.sv
    │   ├── minisoc_scoreboard.sv
    │   └── minisoc_coverage.sv
    │
    ├── refmod/
    │   ├── cpp/
    │   └── aes_lib.svh
    │
    ├── sequences/
    │   ├── spi/
    │   ├── mem/
    │   └── virtual/
    │
    ├── tests/
    │   ├── minisoc_base_test.sv
    │   └── minisoc_nominal_ditl_test.sv
    │
    ├── top_assertions.sv
    └── top_tb.sv
```

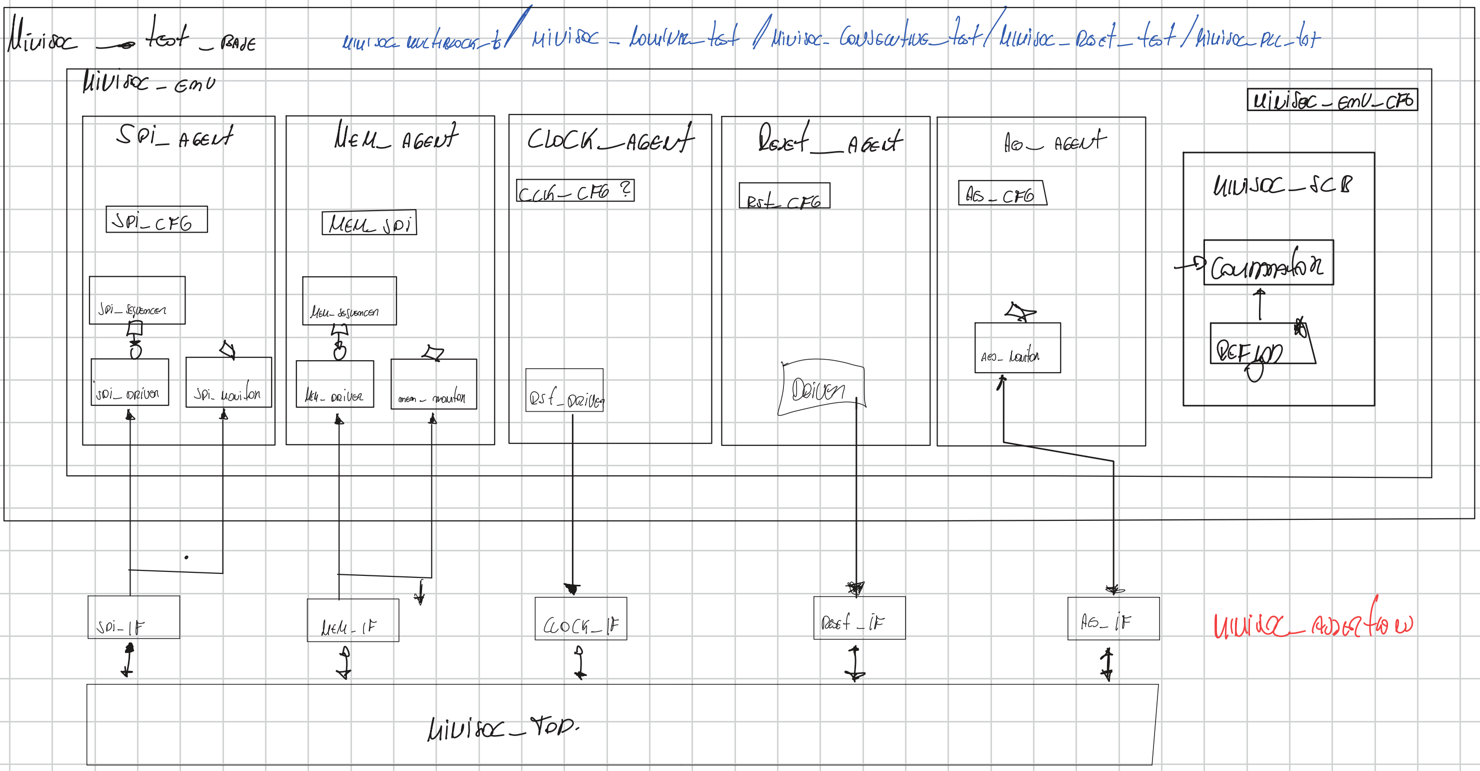
Item_clk
int_period
time_low
time_high
Rand int jitter*

Item_rst
Rand int num
bit_n_high

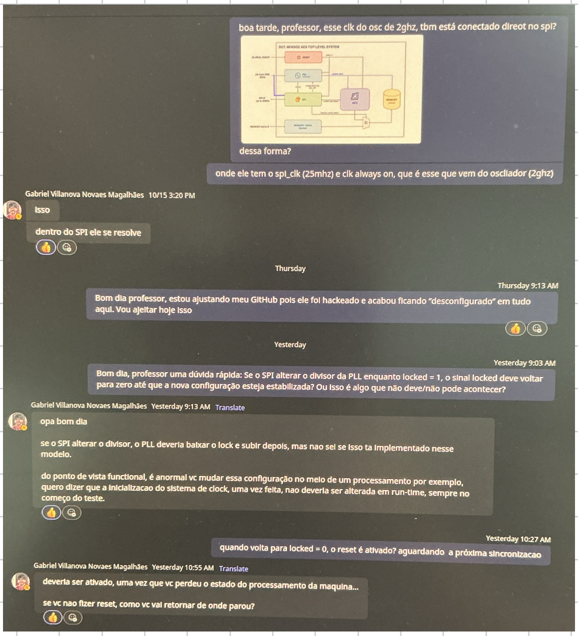
Assertions-

Minisoc top


s
sequences
spi_sequence
s


Minisoc_test_base
minisoc_multiblock_t / minisoc_nominal_test / minisoc_consecutive_test / minisoc_reset_test / minisoc_pll_test
minisoc_env
minisoc_env_cfg
spi_agent
spi_cfg
spi_sequencer
spi_driver
spi_monitor
mem_agent
mem_spi
mem_sequencer
mem_driver
mem_monitor
clock_agent
clk_cfg ?
rst_driver
Reset_agent
rst_cfg
Driver
aes_agent
aes_cfg
aes_monitor
minisoc_scb
comparator
ref_mod
spi_if
mem_if
clock_if
reset_if
aes_if
minisoc_top.
minisoc_assertion

Dúvidas



boa tarde, professor, esse clk do osc de 2ghz, tbm está conectado direot no spi?

dessa forma?

onde ele tem o spi_clk (25mhz) e clk always on, que é esse que vem do oscilador (2ghz)

Gabriel Villanova Novaes Magalhães 10/15 3:20 PM

Isso

dentro do SPI ele se resolve

Thursday

Thursday 9:13 AM

Bom dia professor, estou ajustando meu GitHub pois ele foi hackeado e acabou ficando "desconfigurado" em tudo aqui. Vou ajeitar hoje isso

Yesterday

Yesterday 9:03 AM

Bom dia, professor uma dúvida rápida: Se o SPI alterar o divisor da PLL enquanto locked = 1, o sinal locked deve voltar para zero até que a nova configuração esteja estabilizada? Ou isso é algo que não deve/não pode acontecer?

Gabriel Villanova Novaes Magalhães Yesterday 9:13 AM Translate

opa bom dia

se o SPI alterar o divisor, o PLL deveria baixar o lock e subir depois, mas nao sei se isso ta implementado nesse modelo.

do ponto de vista functional, é anormal vc mudar essa configuração no meio de um processamento por exemplo, quero dizer que a inicializacao do sistema de clock, uma vez feita, nao deveria ser alterada em run-time, sempre no começo do teste.

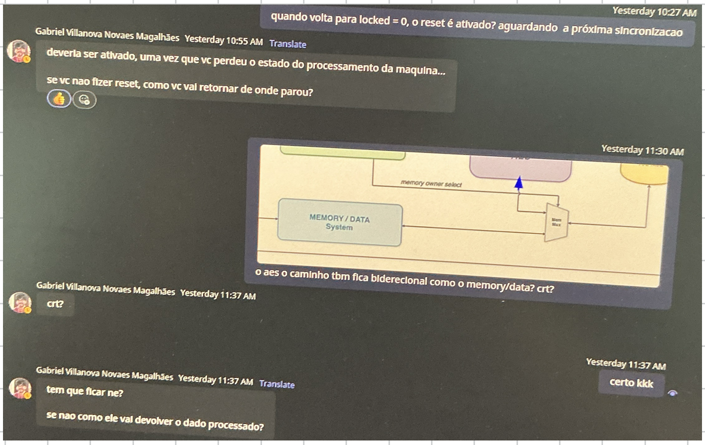
Yesterday 10:27 AM

quando volta para locked = 0, o reset é ativado? aguardando a próxima sincronizacao

Gabriel Villanova Novaes Magalhães Yesterday 10:55 AM Translate

deveria ser ativado, uma vez que vc perdeu o estado do processamento da maquina...

se vc nao fizer reset, como vc vai retornar de onde parou?



Yesterday 10:27 AM

quando volta para locked = 0, o reset é ativado? aguardando a próxima sincronizacao

Gabriel Villanova Novaes Magalhães Yesterday 10:55 AM Translate

deveria ser ativado, uma vez que vc perdeu o estado do processamento da maquina...

se vc nao fizer reset, como vc vai retornar de onde parou?

Yesterday 11:30 AM

memory owner select
MEMORY / DATA System
Mem Mux

o aes o caminho tbm fica bidirecional como o memory/data? crt?

Gabriel Villanova Novaes Magalhães Yesterday 11:37 AM

crt?

Gabriel Villanova Novaes Magalhães Yesterday 11:37 AM Translate

tem que ficar ne?

se nao como ele vai devolver o dado processado?

Yesterday 11:37 AM

certo kkk

Day 3

1. Decisões arquiteturais

Item	Definição
SPI	Scoreboard independente alimentado pelo spi_monitor; modelo simples de registradores, sem RAL inicialmente
Comunicação	uvm_analysis_port, analysis_imp e/ou uvm_tlm_analysis_fifo
Timeouts	Global e individual por operação
Tratamento de falhas	UVM_ERROR para divergências; UVM_FATAL para falhas estruturais
Coverage	Componentes por subsistema integrados no minisoc_env
Regressão	Makefile com testes, seeds e relatórios por fase
Metodologia	Implementação incremental por fase; cada fase deve ser executável e verificável

2. Cinco camadas de verificação

Camada	Componentes e função
L1 - BFMs	clk_driver, rst_driver, spi_driver, mem_driver: estímulos físicos
L2 - Geração / Monitoramento	Sequencers SPI e Memory; monitores SPI, Memory, PLL e AES
L3 - Checking	spi_scoreboard, mem_scoreboard, minisoc_scoreboard, pll_checker e SVA
L4 - Coverage	Cobertura SPI, PLL, Memory, AES, integração e robustez
L5 - Harness / Scenarios	top_tb, minisoc_env, virtual sequences e testes

Clock e Reset: drivers diretamente no minisoc_env, sem agentes nem sequencers. PLL e AES: monitoramento passivo.

3. Matriz de fases

Fase	Escopo	Critério principal
F1 - SPI + PLL	Bring-up, registradores SPI, PLL /2, /4, /8, LOCKED, sys_clk e reset interno	Configurações e comportamento temporal verificados
F2 - Memory	Leitura/escrita RAM, limites, padrões, random e scoreboard	Integridade dos dados e cobertura dos acessos
F3 - AES Functional	Cipher, Inverse Cipher, modos implementados, múltiplos blocos	100% dos cruzamentos válidos
F4 - Integration	Back-to-back, reconfiguração, alternância AES/RAM e transições	100% dos bins válidos de transição
F5 - Robustness & Stress	Reset durante BUSY, escritas protegidas, START concorrente, LOCKED e stress random	Fechamento funcional e ausência de falhas

4. Parâmetros já estabelecidos

Subsistema	Parâmetros
Clock / Reset	Clock de referência: 2 GHz (500 ps); reset ativo baixo por 10 ciclos; clk_cfg e rst_cfg via uvm_config_db
SPI	Mode 0 (CPOL=0, CPHA=0); 25 MHz padrão, configurável via spi_cfg
Transação SPI	spi_item: op, addr, write_data e read_data; resposta separada via get_response()
PLL	Divisores /2, /4 e /8; reset interno AES liberado após 3 ciclos de sys_clk depois de LOCKED
AES	AES-128, Cipher, Inverse Cipher, múltiplos blocos e todos os modos efetivamente implementados
RAM	1024 x 32 bits; acesso externo e AES conforme propriedade da memória

6. Ordem de implementação

Ordem	Entrega
1	Clock e Reset: interfaces, configurações e drivers automáticos
2	SPI Agent: registradores, Mode 0, leitura/escrita e resposta
3	PLL: monitor, checker, SVA, cobertura e teste F1
4	Memory: agente, modelo, scoreboard, cobertura e teste F2
5	AES: monitor, modelo de referência, scoreboard e teste F3
6	Integração: virtual sequences, back-to-back e teste F4
7	Robustez: exceções, stress random, regressão e fechamento F5

7. Pontos a confirmar no RTL antes da codificação

Mapa e larguras dos registradores SPI; formato exato dos quadros; modos AES e codificações; comportamento de condições excepcionais; hierarquia dos sinais internos; temporização e restrições de reconfiguração. Esses pontos não alteram a conclusão do planejamento arquitetural.

Conclusão: Dia 3 concluído em nível de planejamento. Próxima atividade: implementação incremental da F1, iniciando pelos componentes de Clock e Reset.